HOCHSCHULE HEILBRONN

Hochschule Heilbronn

Fakultät Wirtschaft und Verkehr

Master Wirtschaftsinformatik (Business Information Systems)

Winter semester 2026/27

News Sentiment Analysis by URL

Executive Pitch

Teofil Wetzel, 191650

twetzel@stud.hs-heilbronn.de

Nick Fronius, 215785

nfronius@stud.hs-heilbronn.de

Module:
AI Operations

Lecturer:
Pranav Sharma

Date:
October 29, 2026

1 Problem

Judging the sentiment of news articles by hand is slow, expensive and subjective. Newsrooms, PR teams and analysts have to screen large numbers of articles every day and decide whether the coverage is negative, neutral or positive. Different readers often reach different conclusions, and there is no traceable, repeatable assessment.

Our business goal in the AI Operations module is therefore: given a URL or free text (German and English), automatically return one of three classes — negative, neutral or positive. The use case is media monitoring and reputation tracking: whoever sees quickly how a topic is coloured in the coverage can react earlier and use resources in a more targeted way.

We framed the project as an MLOps problem on purpose. Training a model once is not enough. We need a reproducible pipeline from the raw article to the API response, a fair model comparison on the same test set, and traceable experiment documentation.

2 Stakeholders

Three user groups benefit directly. **Newsrooms** can triage incoming articles faster and prioritise topics without reading every text by hand. **PR & Communications** teams get a signal on their reputation: if negative reports about a brand or an event pile up, this becomes visible early. **Analysts** use sentiment as an additional indicator in the news flow, for example for market or topic monitoring.

What we deliver is a service interface and evidence that it holds up. The FastAPI endpoints accept either text or a URL (XOR): for a URL, the article body is extracted with Trafilatura, preprocessed and classified. The response contains the predicted class. In parallel, we document the model comparison in MLflow, so that the choice of the production model follows from measurable metrics rather than gut feeling.

3 Methods

The pipeline has three steps: scraping or text input, preprocessing (cleaning, tokenisation per model family) and classification into three sentiment classes. The lead metric is macro-F1, so that no class is systematically neglected. Both iterations remain runnable in the repository: the first in `src/` and `app/`, the second as the package `second_path`.

In the first iteration, five approaches are evaluated on the same financial-sentence test set (PhraseBank English and Kenpache German, split 70 % / 15 % / 15 %, seed 42, $n = 1{,}270$):

- **TF-IDF + Logistic Regression** as the classic baseline (macro-F1 0.6562),

- **TF-IDF + LinearSVC** as the best score on this test (macro-F1 0.7202, about 0.02 ms per text, roughly 4 MB),
- **`tabularisai/multilingual-sentiment-analysis`** as a Hugging Face zero-shot model (macro-F1 0.3636),
- **`cardiffnlp/twitter-xlm-roberta-base-sentiment`**, also as a zero-shot comparison (macro-F1 0.4859),
- **DistilBERT fine-tuning**, measured as a CPU subset (2,000 sentences, one epoch, context 128; macro-F1 0.6142). The full GPU run in the Colab notebook is described but not measured.

The second iteration replaces the word-count vectors with the multilingual encoder `paraphrase-multilingual-MiniLM-L12-v2`. First a frozen head (macro-F1 0.6656 on its own test set with $n = 1{,}787$), then exactly one fine-tuning of all encoder weights: three epochs, batch 32, context 128, BF16, AdamW, class-weighted cross-entropy. This raises the same test to macro-F1 0.7187. After that come separate ablations within the train split (source-group CV, seed 20261019), without reopening the old test set: R1 weights by language and class, R2 changes chunk aggregation and HTML extraction, R3 stores and computes in BF16, R4 combines the three. R3 is shipped because its CV mean of 0.6729 lies within the pre-defined 0.005 of the best, R4 (0.6740), and the tie rule picks the lower latency (2.381 ms vs. 2.419 ms) at about 241 instead of 465 MiB.

Serving and operations: FastAPI with `/predict` (text or URL, Trafilatura in the first iteration), GitHub Actions for both test paths, MLflow store `./mlruns` with the tags `iteration=1-first-path` and `iteration=2-second-path`. Repository: https://github.com/Rasvan-Teofil/ai-operations-projekt.

4 From the first to the second iteration

In the first iteration, the classic LinearSVC was strong on its financial-sentence test. The transformer attempts were not: zero-shot stays at 0.36 and 0.49 macro-F1, CPU fine-tuning at 0.61, below both TF-IDF models. Performance on more general news was therefore not shown, and the full DistilBERT run was missing.

In the second iteration we changed the approach, not just one number. Instead of TF-IDF a multilingual MiniLM, instead of a short CPU subset a fixed fine-tuning on the full train split, followed by single parameter and operations changes under a selection rule fixed in advance. On the second iteration's own test set, fine-tuning improves on the frozen encoder by 0.0531 macro-F1 (0.6656 to 0.7187) at practically the same inference time. The source CV in Table 1 shows where the change helps and where it does not: the jump lies between the frozen baseline B0 (0.6306) and the reference

fine-tune R0 (0.6724, $\Delta + 0.0418$). R1 to R4 only move the CV mean in the third decimal place. The clear operational gain of R3 is the halved artefact size.

Table 1: Source CV in the second iteration, mean over three folds.

Stage	Macro-F1	DE	EN	MiB
B0 frozen	0.6306	0.6566	0.6059	465.1
R0 reference fine-tune	0.6724	0.7053	0.6425	465.1
R1 language×class	0.6739	0.7076	0.6430	465.1
R2 article processing	0.6727	0.7053	0.6430	465.1
R3 BF16, selected	0.6729	0.7062	0.6427	240.7
R4 combination	0.6740	0.7065	0.6441	240.7

These CV figures and the two own test sets (0.7202 and 0.7187) are not one shared ranking. The comparison that sees both approaches on the same texts is the external audit run after the selection had already been frozen: 600 English SEntFiN headlines with a unanimous entity label and their German OPUS-MT translation. German is a transfer check there, not a native gold test. Table 2 and Figure 1 reproduce the values from `reports/development/branch_comparison.md`.

Table 2: Shared external benchmark on 600 headline pairs.

Solution	Macro-F1 EN	Macro-F1 DE	ms/text	MiB
First iteration, reconstructed	0.4524	0.4293	0.250	4.08
Second iteration v1	0.7360	0.6458	2.335	465.14
Second iteration v2 (R3)	0.7293	0.6557	2.326	240.73

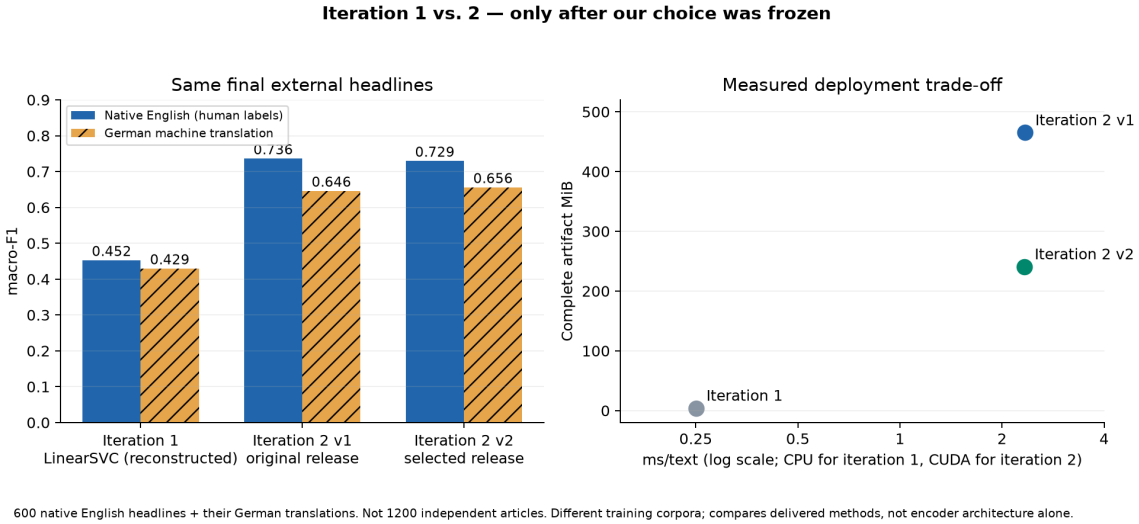


Figure 1: External comparison on the 600 headline pairs.

On the native English headlines, v2 is 0.2770 macro-F1 ahead of the reconstructed first default. A paired bootstrap over these 600 already predicted headlines (1,000 resamples, seed 20261019) gives a 95 % percentile interval for the difference of 0.2257 to 0.3249. v2 is not an across-the-board accuracy gain over v1: English goes from 0.7360 to 0.7293, translated German from 0.6458 to 0.6557. The gain of v2 is the smaller package at similar CV quality. The first default remains much smaller and faster on the CPU. Because the training corpora differ, the audit tests delivered approaches and does not prove an isolated architectural advantage.

5 Next steps

The second iteration is measured and documented next to the first in the repository. The full DistilBERT GPU run of the first path is still open; until it is on the test set with $n = 1{,}270$, LinearSVC remains the evidenced choice there. R2 and R4 pass the HTML fixtures (six of six instead of two of six), but they are not the API default chosen by the latency rule.

The error costs of a false alarm and of a missed negative report are still not quantified. The training domain remains financial sentences; general coverage, irony and long articles are not covered by evidence. A bias analysis by language or topic remains a lower priority. The local MLflow runs are sufficient for the model choice: live via `python -m src.train`, and the already recorded GPU metrics without retraining via `python -m src.import_recorded`, both in the store `./mlruns`.

In summary: the first iteration delivers a fast, small default and shows that zero-shot and the short CPU fine-tuning do not beat it. The second iteration improves encoder quality on the shared headline benchmark and halves the artefact size with BF16, without redefining the selection rule afterwards.